國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab07

姓名：李品融

學號：113310217

Lab07-Q1
這題在練習 Stack 結構的應用，透過補足 precedence 與 InfixToPostfix 函數，將中序表達式轉換為後序表達式，並處理運算子優先權與括號邏輯。

Code

```
#include <iostream>
#include <cctype> // 提供 isalnum() 函數，用來判斷字元是否為數字或英文字母
#include <cstring> // 提供字串處理相關功能
using namespace std;

// 定義 Stack 的節點結構
struct Node {
 char data;  // 存儲字符 (運算子或括號)
 Node* next; // 指向下一個節點
};

// 使用 linked list 實作 Stack
class Stack {
private:
 Node* top; // 指向堆疊頂端
public:
 Stack() { top = NULL; } // 初始化堆疊

 // Push 操作：將元素放入堆疊
 void push(char ch) {
  Node* newNode = new Node; // 創建新節點
  newNode->data = ch;       // 存入數據
  newNode->next = top;      // 讓新節點指向目前的頂端
  top = newNode;            // 更新頂端指標
 }

 // Pop 操作：移除並回傳頂端元素
 char pop() {
  if (isEmpty()) return '\0'; // 若堆疊為空，回傳空字符
   char ch = top->data;         // 取得頂端元素
   Node* temp = top;            // 暫存頂端節點
   top = top->next;             // 移動頂端指標
   delete temp;                 // 釋放記憶體
```

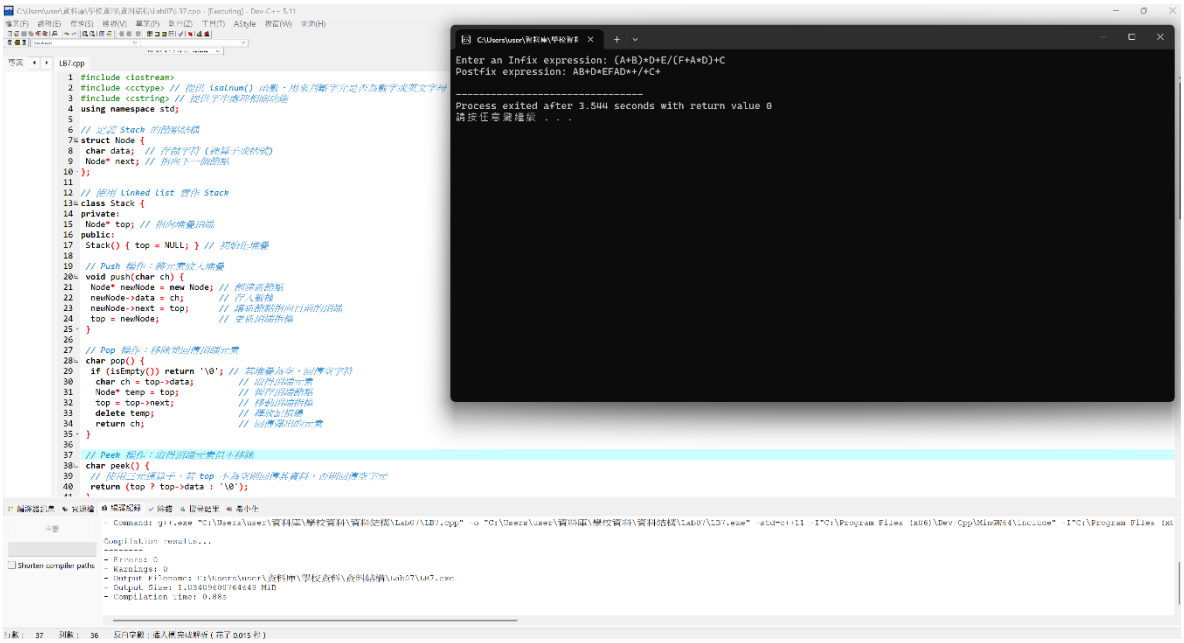
```
    return ch;                    // 回傳彈出的元素
  }

  // Peek 操作：取得頂端元素但不移除
  char peek() {
   // 使用三元運算子，若 top 不為空則回傳其資料，否則回傳空字元
   return (top ? top->data : '\0');
  }

  // 判斷堆疊是否為空
  bool isEmpty() {
   return top == NULL; //如果 top 扔指向 NULL，表示堆疊內沒有任何節點
  }
};

// 判斷運算子(加減乘除) 的優先順序
int precedence(char op) {
  // 加法、減法權重最低，設為 1
  if (op == '+' || op == '-') {
   return 1;
  }
  // 乘法、除法權重較高，設為 2
  if (op == '*' || op == '/') {
   return 2;
  }
  // 若非運算子 (括號、非法字元) 則回傳 0
  return 0; // 對於括號或其他字元回傳 0
}

// 將中序表達式 (infix) 轉換為後序表達式 (postfix)
void InfixToPostfix(const char* infix, char* postfix) {
  Stack s; // 宣告一個堆疊物件，用於暫存尚未處理的運算子與括號
  int j = 0; // 用於填寫 postfix 陣列的索引

  // 迴圈讀取中序字串，直到字串結束符號 '\0'
  for (int i = 0; infix[i] != '\0'; i++) {
   char c = infix[i]; // 取得目前處理中的字元
```

```
  // 如果是運算元 (字母或數字)，直接輸出到 postfix
  if (isalnum(c)) {
   postfix[j++] = c; // 直接將運算元輸出到後序字串中
  }

  // 如果是左括號 '('，推入 Stack
  else if (c == '(') {
   s.push(c); // 直接堆入堆疊，作為後續運算子的邊界
  }

  // 如果是右括號 ')'，彈出 Stack 直到遇見 '('
  else if (c == ')') {
   // 不斷彈出堆疊頂端的運算子並輸出，直到遇見左括號為止
   while (!s.isEmpty() && s.peek() != '(') {
    postfix[j++] = s.pop();
   }
   s.pop(); // 將最後匹配到的 '(' 從堆疊中彈出並丟棄 (不輸出到結果)
  }

  // 如果是運算子 (+、 -、 *、 /)
  else {
   // 當 Stack 不為空，且堆疊頂端運算子優先權大於或等於當前運算子的優先權時
   while (!s.isEmpty() && precedence(s.peek()) >= precedence(c)) {
    postfix[j++] = s.pop();
   }
   s.push(c); // 處理完優先權較高的運算子後，將當前運算子推入 Stack
  }
 }

 // 當中序字串掃描完畢，將 Stack 中剩下的運算子全部彈出
 while (!s.isEmpty()) {
  postfix[j++] = s.pop();
 }

 postfix[j] = '\0'; // 在字串結尾加上結束符號
}
```

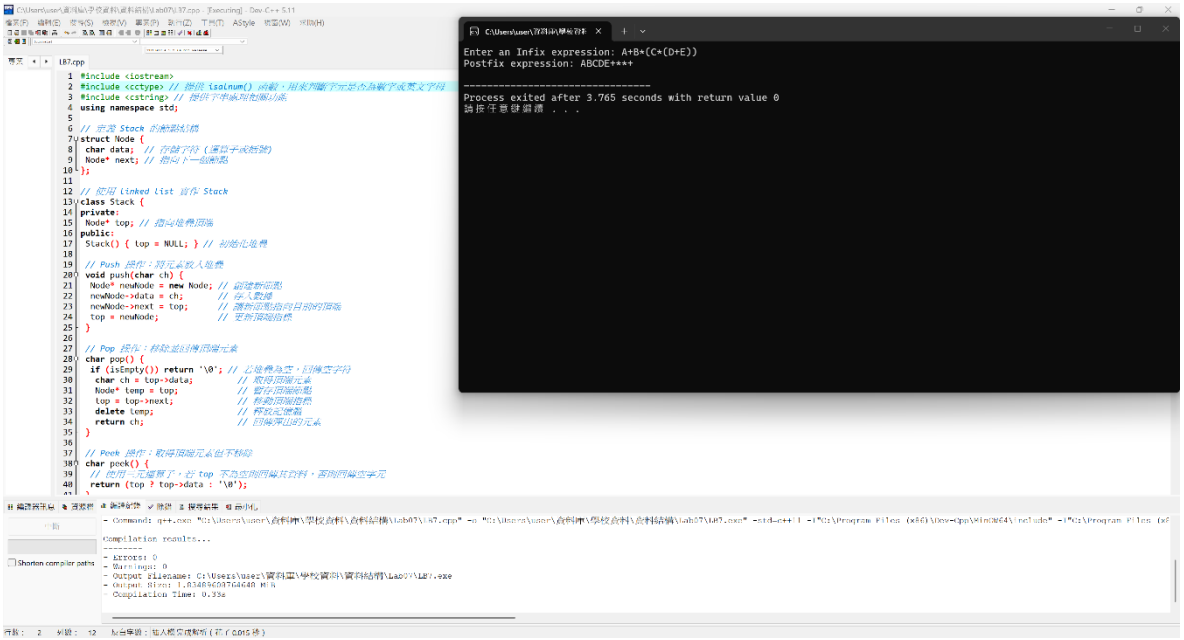
```cpp
int main() {
 char infix[100], postfix[100];
 cout << "Enter an Infix expression: ";
 cin >> infix; // 輸入中序表達式

 InfixToPostfix(infix, postfix); // 轉換為後序表達式
 cout << "Postfix expression: " << postfix << endl; // 輸出後序表達式

 return 0;
}
```

Result

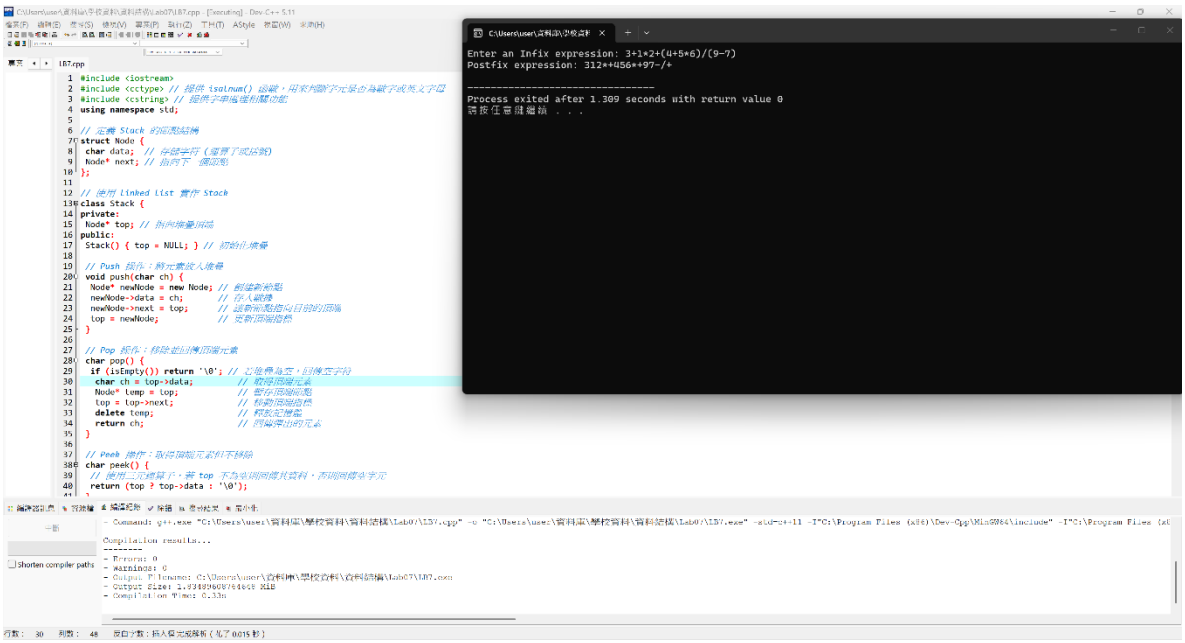
範例一:輸入(A+B)*D+E/(F+A*D)+C，輸出 AB+D*EFAD*+/+C+



範例二:輸入 A+B*(C*(D+E))，輸出 ABCDE+**+



範例三: 3+1*2+(4+5*6)/(9-7)，312*+456*+97-/+



Lab07-Q3

透過繪製表格，逐步填寫輸入字元、Stack 內容、Top 指標及輸出結果，來了解並掌握括號運算子的優先權邏輯。

Result

範例一:輸入(A+B)*D+E/(F+A*D)+C，輸出 AB+D*EFAD*+/+C+

Token	Stack[0]	Stack[1]	Stack[2]	Stack[3]	Stack[4]	Top	Output
(	(					0	
A	(					0	A
+	(	+				1	A
B	(	+				1	AB
)						-1	AB+
*	*					0	AB+
D	*					0	AB+D
+	+					0	AB+D*
E	+					0	AB+D*E
/	+	/				1	AB+D*E
(	+	/	(			2	AB+D*E
F	+	/	(			2	AB+D*EF
+	+	/	(	+		3	AB+D*EF
A	+	/	(	+		3	AB+D*EFA
*	+	/	(	+	*	4	AB+D*EFA
D	+	/	(	+	*	4	AB+D*EFAD
)	+	/				1	AB+DEFAD+
+	+					0	AB+DEFAD+/+
C	+					0	AB+DEFAD+/+C
<eos>						-1	AB+DEFAD+/+C+

範例二:輸入 A+B*(C*(D+E))，輸出 ABCDE+**+

Token	Stack[0]	Stack[1]	Stack[2]	Stack[3]	Stack[4]	Stack[5]	Top	Output
A							-1	A
+	+						0	A
B	+						0	AB
*	+	*					1	AB
(	+	*	(				2	AB
C	+	*	(				2	ABC
*	+	*	(	*			3	ABC
(	+	*	(	*	(		4	ABC
D	+	*	(	*	(		4	ABCD
+	+	*	(	*	(	+	5	ABCD
E	+	*	(	*	(	+	5	ABCDE
)	+	*	(	*			3	ABCDE+
)	+	*					1	ABCDE+*
<eos>							-1	ABCDE+**+

範例三: 3+1*2+(4+5*6)/(9-7)，312*+456*+97-/+

Token	Stack[0]	Stack[1]	Stack[2]	Stack[3]	Top	Output
3					-1	3
+	+				0	3
1	+				0	31
*	+	*			1	31
2	+	*			1	312
+	+				0	312*+
(	+	(			1	312*+
4	+	(			1	312*+4
+	+	(	+		2	312*+4
5	+	(	+		2	312*+45
*	+	(	+	*	3	312*+45
6	+	(	+	*	3	312*+456
)	+				0	312*+456*+
/	+	/			1	312*+456*+
(	+	/	(		2	312*+456*+
9	+	/	(		2	312*+456*+9
-	+	/	(	-	3	312*+456*+9
7	+	/	(	-	3	312*+456*+97
)	+	/			1	312*+456*+97-
<eos>					-1	312*+456*+97-/+